


**M. H. SABOO SIDDIK COLLEGE OF ENGINEERING**

8, Saboo Siddik Polytechnic road, Byculla, Mumbai-8
**Department of Computer Science and Engineering (AI & ML)**

# DATABASE MANAGEMENT SYSTEM

***S.E. – CSE (AIML)***

***SEM- IV***

***COURSE CODE : 2164112***

**Dr. Muhib A. Lambay**
**Asst. Professor**

# Module 1:
# Introduction to Databases

# What is Data?

- Data is a collection of a distinct small piece of information. It can be in variety of forms like text, numbers etc. and can be stored in pieces of paper or electronic memory, etc.

- Word **Data** means single piece of information.

- Data becomes **information** when it is processed, turning it into something meaningful, that is information, derived from the data collected.

# What is Database?

- A database is an organized collection of data, so that it can be easily accessed and managed.

- You can organize data into tables, rows, columns, and index it to make it easier to find relevant information.

- The main purpose of the database is to operate a large amount of information by storing, retrieving, and managing data.

# What is Database?

- There are many websites nowadays managed through databases. For example, a website that checks the availability of rooms in a hotel.

- There are many databases (softwares) available like MySQL, Oracle, MongoDB, SQL Server, **XAMPP Server** etc.

- Databases are managed by the database management system (DBMS).

# What is DBMS?

- DBMS is a software that allows creation, definition and manipulation of database, allowing users to store, process and analyze data easily.

- DBMS provides us with an interface to perform various operations like creating database, storing data in it, updating data, creating tables in the database and a lot more.

- DBMS also provides protection and security to the databases.

- SQL or Structured Query Language is used to operate on the data stored in a database.

# Database Applications – DBMS

- **Telecom**: There is a database to keeps track of the information regarding calls made, network usage, customer details etc. Without the database systems it is hard to maintain that huge amount of data that keeps updating every millisecond.

- **Airlines/Railways**: To travel though airlines/railways, we make reservations, this reservation information along with flight/train schedule is stored in database.

- **Online shopping**: You must be aware of the online shopping websites such as Amazon, Flipkart etc. These sites store the product information, your addresses and preferences, credit details and provide you the relevant list of products based on your query. All this involves a Database management system.

# Database Applications – DBMS

- **Library Management System**: There are many books in the library so; it is difficult to store the record of books in a register.(DBMS) is utilized to keep up all the data identified with the name of the book, issue date, accessibility of the book, and its writer.

- **Social Media Sites**: Every day, many people group pursue online media accounts like Pinterest, Facebook, Twitter, and Google.

- **Healthcare System**: DBMS is used in healthcare to manage patient data, medical records, and billing information.

- **Security**: DBMS provides security features to ensure that only authorized users have access to the data.

## Characteristics/Advantages of Database Management System

- **Data stored into Tables**: Data is never directly stored into the database. Data is stored into tables, created inside the database. DBMS also allows to have relationships between tables which makes the data more meaningful and connected.

- **Reduced Redundancy: Normalization (Minimum Redundancy).**

- **Data Consistency**: On Live data, i.e. data that is being continuously updated and added, maintaining the consistency of data can become a challenge. But DBMS handles it all by itself.

# Characteristics of Database Management System

- **Support Multiple user and Concurrent Access:** DBMS allows multiple users to work on it(update, insert, delete data) at the same time and still manages to maintain the data consistency.

- **Query Language**: DBMS provides users with a simple Query language, using which data can be easily fetched, inserted, deleted and updated in a database.

- **Security**: The DBMS also takes care of the security of data, protecting the data from un-authorised access. In a typical DBMS, we can create user accounts with different access permissions, using which we can easily secure our data by restricting user access.

# Characteristics of Database Management System

- **Easy access to data** – Database systems manages data in such a way so that the data is easily accessible with fast response times.

- **Easy recovery**: Since database systems keeps the backup of data, it is easier to do a full recovery of data in case of a failure.

# Disadvantages of DBMS

- Database design is complex, difficult and time consuming.

- Except MySQL, which is open source, licensed DBMSs are generally costly.

- Large amount of investment is needed to setup the required hardware or to repair software failure.

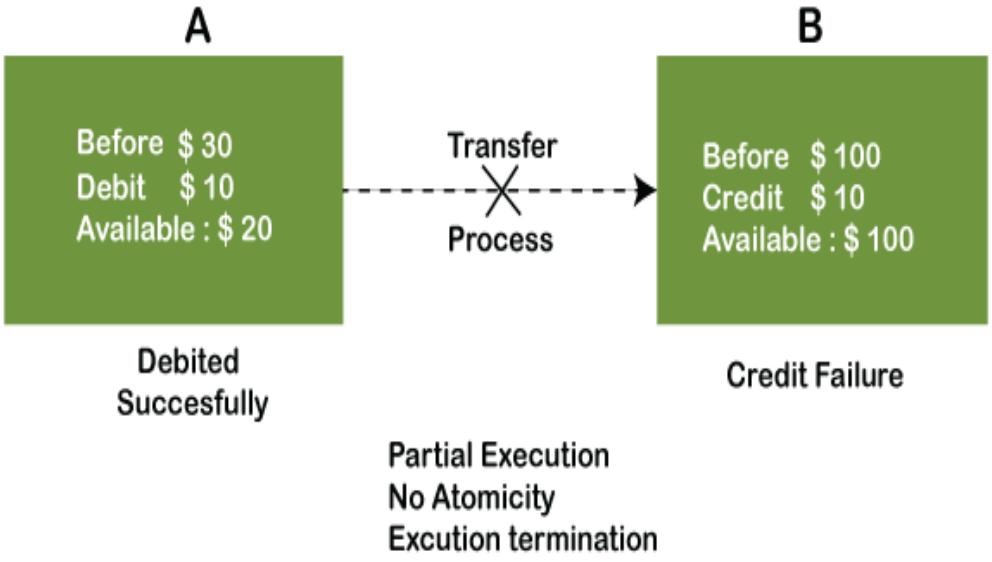
- For designing and maintaining the database systems, the people need to be trained.

- They are large in size.

# ACID Properties

**1) Atomicity:** The term atomicity defines that the data remains atomic. It means if any operation is performed on the data, either it should be performed or executed completely or should not be executed at all. It further means that the operation should not break in between or execute partially. In the case of executing operations on the transaction, the operation should be completely executed and not partially.

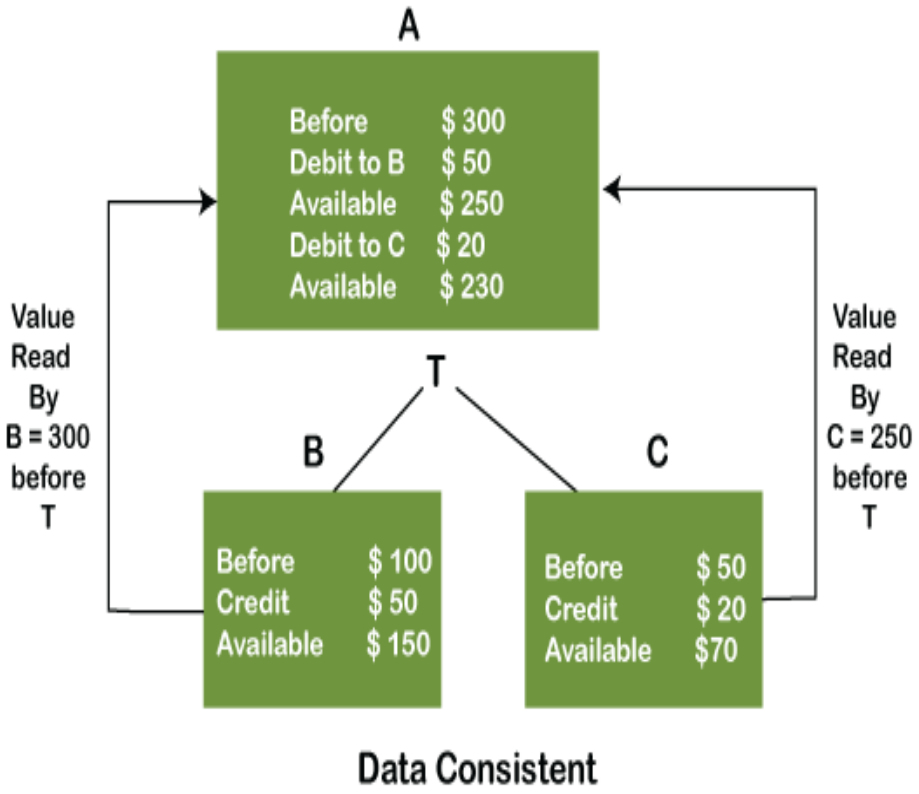
**Example:** If Remo has account A having $30 in his account from which he wishes to send $10 to Sheero's account, which is B. In account B, a sum of $ 100 is already present. When $10 will be transferred to account B, the sum will become $110. Now, there will be two operations that will take place. One is the amount of $10 that Remo wants to transfer will be debited from his account A, and the same amount will get credited to account B, i.e., into Sheero's account. Now, what happens - the first operation of debit executes successfully, but the credit operation, however, fails. Thus, in Remo's account A, the value becomes $20, and to that of Sheero's account, it remains $100 as it was previously present.



# ACID Properties

**2) Consistency:** The word **consistency** means that the value should remain preserved always. In DBMS, the integrity of the data should be maintained, which means if a change in the database is made, it should remain preserved always. In the case of transactions, the integrity of the data is very essential so that the database remains consistent before and after the transaction. The data should always be correct.

**Example:**

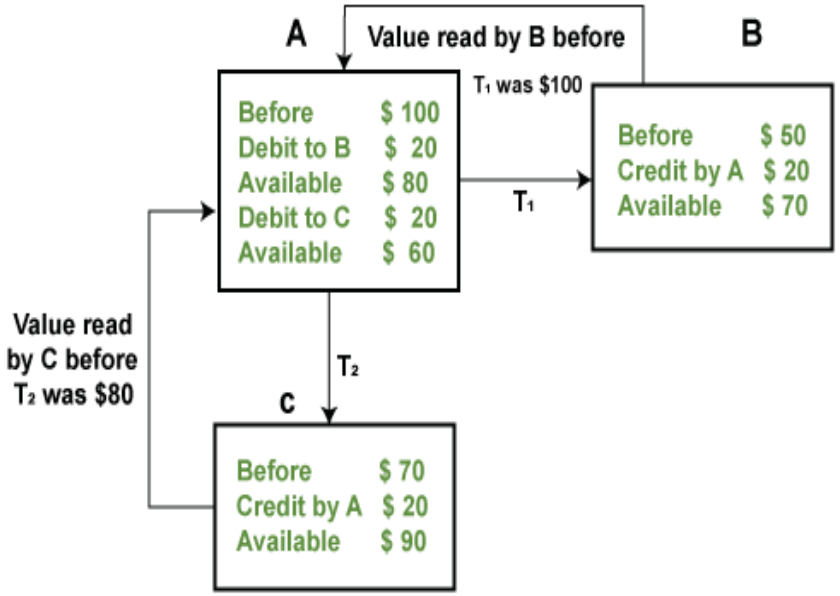



Data Consistent

# ACID Properties

**4) Isolation:** The term 'isolation' means separation. In DBMS, Isolation is the property of a database where no data should affect the other one and may occur concurrently. In short, the operation on one database should begin when the operation on the first database gets complete. It means if two operations are being performed on two different databases, they may not affect the value of one another. In the case of transactions, when two or more transactions occur simultaneously, the consistency should remain maintained. Any changes that occur in any particular transaction will not be seen by other transactions until the change is not committed in the memory.

**Example:** If two operations are concurrently running on two different accounts, then the value of both accounts should not get affected. The value should remain persistent. As you can see in the below diagram, account A is making T1 and T2 transactions to account B and C, but both are executing independently without affecting each other. It is known as Isolation.




Isolation - Independent execution of $T_1$ & $T_2$ by A

• **Isolation.** A transaction is isolated from all other transactions. Transactions can run concurrently only if they don't interfere with each other. Returning to the transfer transaction example, if another transaction were to attempt to withdraw funds from the same savings account, isolation would prevent the second transaction from firing. Without isolation, it might be possible for the second transaction to withdraw more funds than are available in the account after the first transaction was completed.

# ACID Properties

**4) Durability:** Durability ensures the permanency of something. In DBMS, the term durability ensures that the data after the successful execution of the operation becomes permanent in the database. The durability of the data should be so perfect that even if the system fails or leads to a crash, the database still survives. However, if gets lost, it becomes the responsibility of the recovery manager for ensuring the durability of the database. For committing the values, the COMMIT command must be used every time we make changes.

Therefore, the ACID property of DBMS plays a vital role in maintaining the consistency and availability of data in the database.

**Example: After successfully transferring money from Account A to Account B, the changes are stored on disk. Even if there is a crash immediately after the commit, the transfer details will still be intact when the system recovers, ensuring durability.**

# DBMS vs. File System

| Database Management System | File System |
|---|---|
| Multi-user access | It does not support multi-user access |
| Design to fulfill the need for small and large businesses | It is only limited to smaller DBMS system |
| Easy to implement complicated transactions | No support for complicated transactions |
| Remove Data Redundancy and Data Inconsistency | Redundancy issues |
| Database systems define the data in a structured manner. Also there is well defined co-relation among the data. | File systems define the data in un-structured manner. |
| Data Security | No Security |
| Easy Recovery | Updated data not recovered |
| Easy access to data | Data accessing can be difficult |
| Highly structured format with tables, rows, and columns | Simple, unstructured format with records stored in plain text files |
| Supports ACID properties | ACID properties not supported |

# DBMS Architecture

- The architecture of DBMS depends on the computer system on which it runs. For example, in a client-server DBMS architecture, the database systems at server machine can run several requests made by client machine.

- There are three types of DBMS architecture:
  - Single tier architecture
  - Two tier architecture
  - Three tier architecture

# 1-Tier Architecture

- In this architecture, the database is directly available to the user. It means the user can directly sit on the DBMS and uses it. Any changes done here will directly be done on the database itself.

- In 1-Tier Architecture, the user works directly with the database on the same system. This means the client, server, and database are all located on one machine.

- The user can open the application, interact with the data, and perform tasks **without** needing a separate server or **network connection**.

# 1-Tier Architecture

- For example, lets say you want to fetch the records of employee from the database and the database is available on your computer system, so the request to fetch employee details will be done by your computer and the records will be fetched from the database by your computer as well. This type of system is generally referred as local database system.

# 1-Tier Architecture




1-tier Architecture Diagram

The simplest of Database Architecture are **1 tier** where the Client, Server, and Database all reside on the same machine.

Anytime you install a DB in your system and access it to practice SQL queries it is 1 tier architecture.

## Advantages of 1-Tier Architecture

Below mentioned are the advantages of 1-Tier Architecture.

- **Simple Architecture:** 1-Tier Architecture is the most simple architecture to set up, as only a single machine is required to maintain it.
- **Cost-Effective:** No additional hardware is required for implementing 1-Tier Architecture, which makes it cost-effective.
- **Easy to Implement:** 1-Tier Architecture can be easily deployed, and hence it is mostly used in small projects.

## Disadvantages of 1-Tier Architecture

- **Limited to Single User:** Only one person can use the application at a time. It's not designed for multiple users or teamwork.
- **Poor Security:** Since everything is on the same machine, if someone gets access to the system, they can access both the data and the application easily.
- **No Centralized Control:** Data is stored locally, so there's no central database. This makes it hard to manage or back up data across multiple devices.

# 2-Tier Architecture

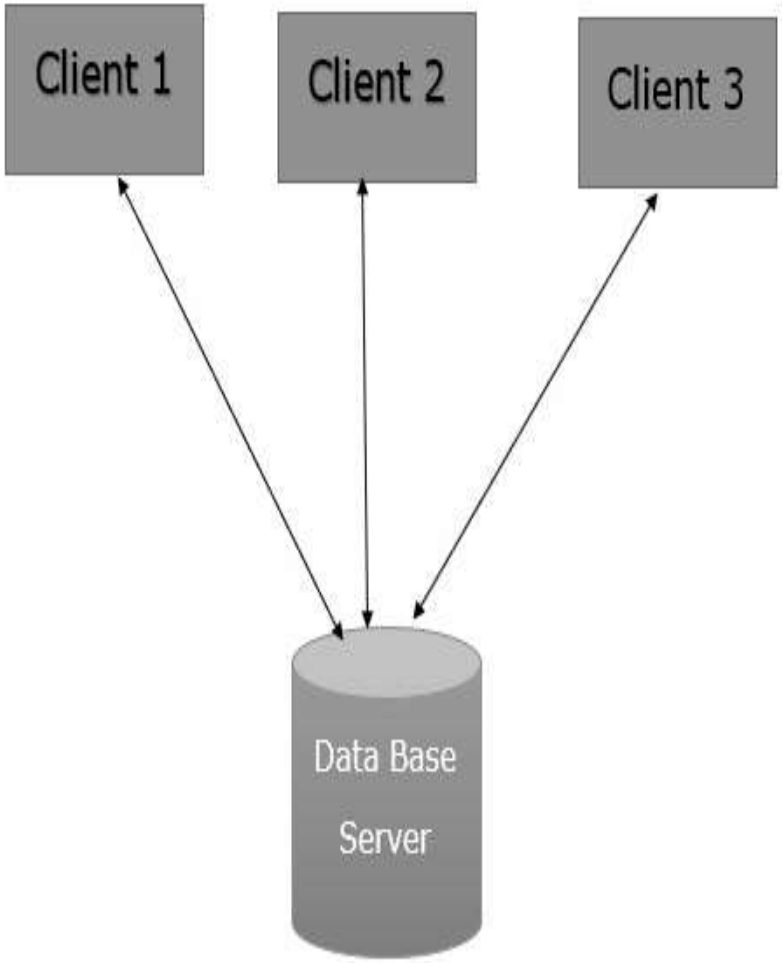
- The 2-Tier architecture is same as basic client-server. In the two-tier architecture, applications on the client end can directly communicate with the database at the server side.

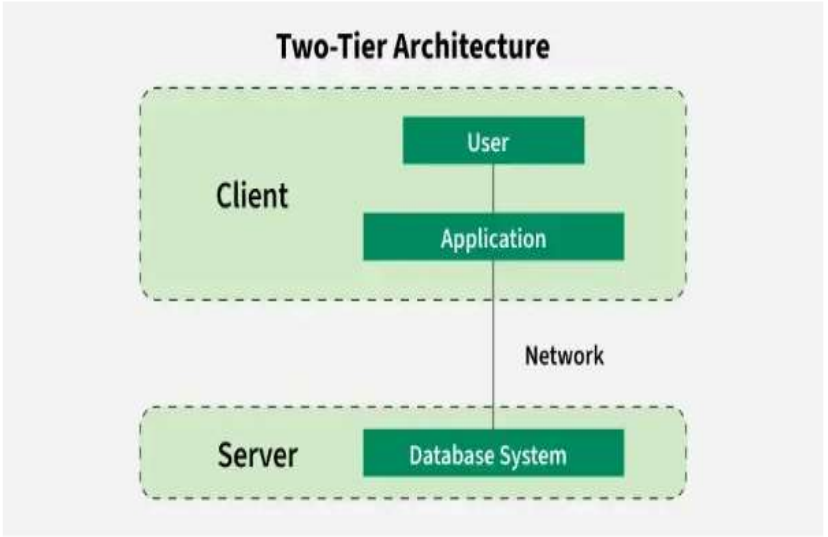
- The user interfaces and application programs are run on the client-side.

- The server side is responsible to provide the functionalities like: query processing and transaction management.

- To communicate with the DBMS, client-side application establishes a connection with the server side.

# 2-Tier Architecture




DBMS 2-Tier Architecture

## Advantages of 2-Tier Architecture

- **Easy to Access:** 2-Tier Architecture makes easy access to the database, which makes fast retrieval.
- **Scalable:** We can scale the database easily, by adding clients or upgrading hardware.
- **Low Cost:** 2-Tier Architecture is cheaper than 3-Tier Architecture and Multi-Tier Architecture.
- **Easy Deployment:** 2-Tier Architecture is easier to deploy than 3-Tier Architecture.
- **Simple:** 2-Tier Architecture is easily understandable as well as simple because of only two components.

## Disadvantages of 2-Tier Architecture

- **Limited Scalability:** As the number of users increases, the system performance can slow down because the server gets overloaded with too many requests.
- **Security Issues:** Clients connect directly to the database, which can make the system more vulnerable to attacks or data leaks.
- **Tight Coupling:** The client and the server are closely linked. If the database changes, the client application often needs to be updated too.

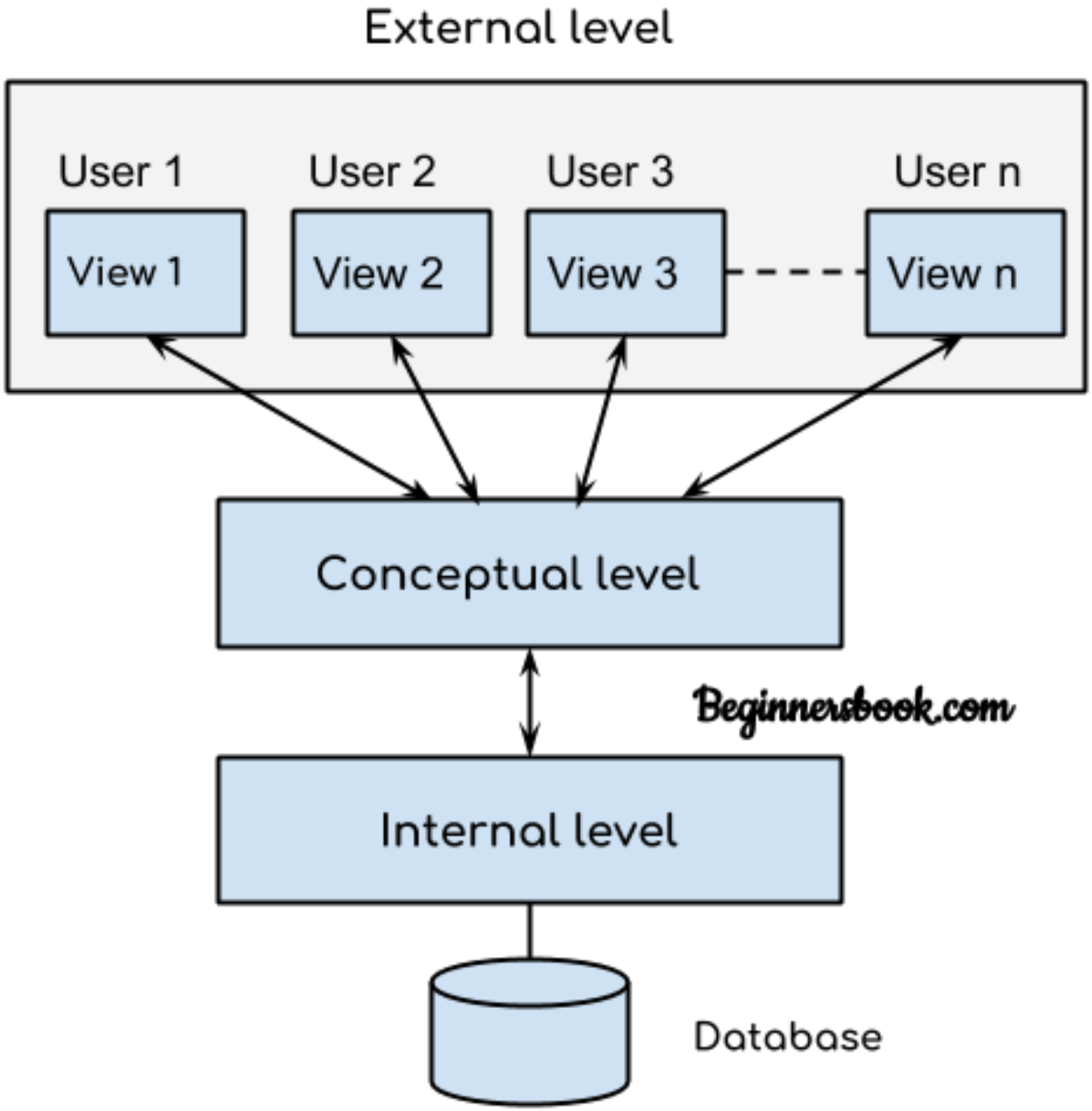
# 3-Tier Architecture

In 3-Tier Architecture, there is another layer between the client and the server. The client does not directly communicate with the server. Instead, it interacts with an application server which further communicates with the database system and then the query processing and transaction management takes place.

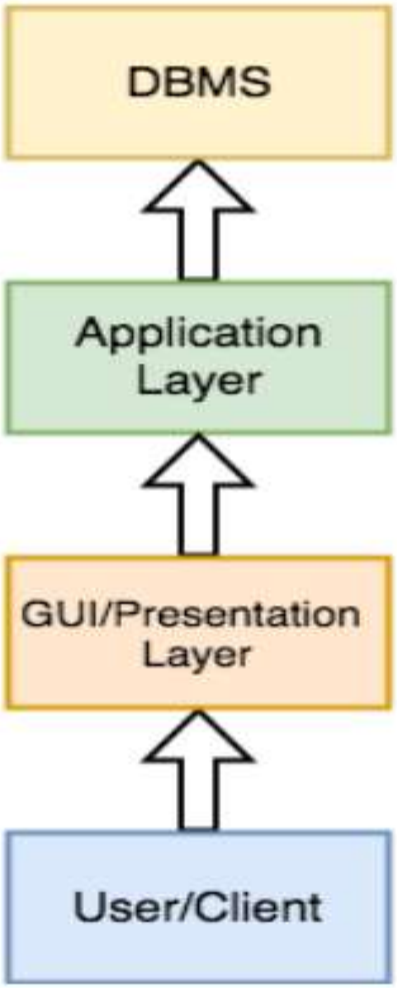
This intermediate layer acts as a medium for the exchange of partially processed data between the server and the client. This type of architecture is used in the case of large web applications.

**Example:** E-commerce Store

- **User**: You visit an online store, search for a product and add it to your cart.
- **Processing**: The system checks if the product is in stock, calculates the total price and applies any discounts.
- **Database**: The product details, your cart and order history are stored in the database for future reference.


External level
User 1
User 2
User 3
User n
View 1
View 2
View 3
View n
Conceptual level
Beginnersbook.com
Internal level
Database

# 3-Tier Architecture




In three-tier architecture, another layer is present between the client machine and server machine.

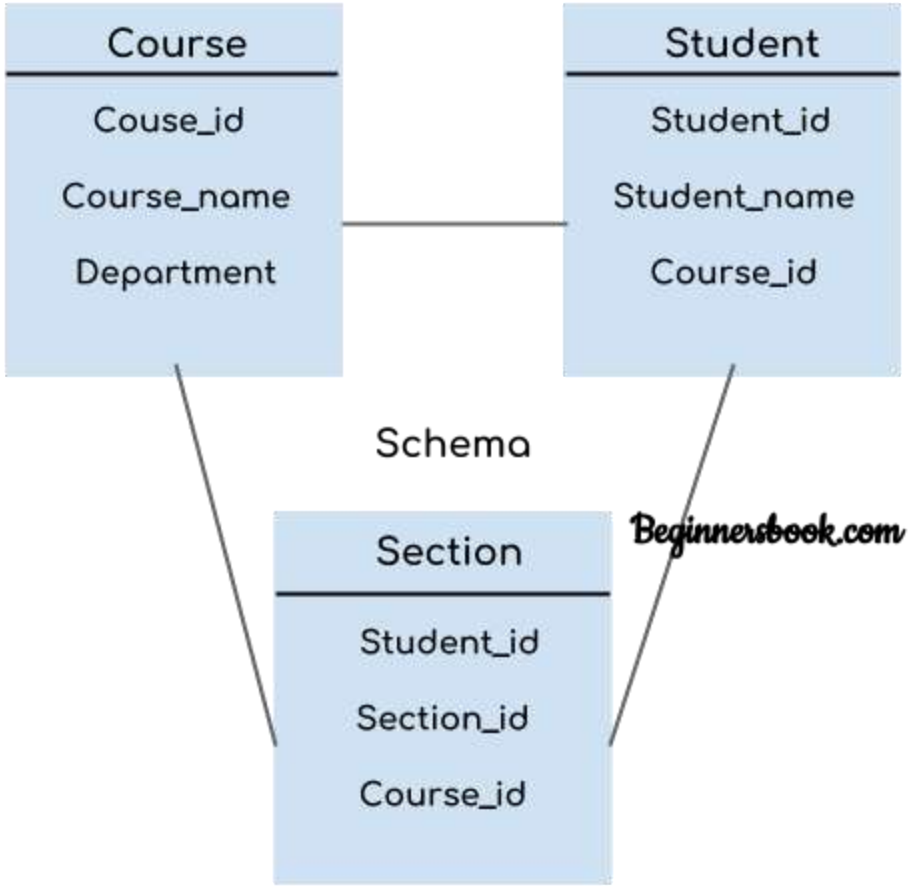
In this architecture, the client application doesn't communicate directly with the database systems present at the server machine, rather the client application communicates with server application and the server application internally communicates with the database system present at the server.

# Database Schema/Overall Structure of DBMS

- Design of a database is called the schema. For example: An employee table in database exists with the following attributes:

- EMP_NAME EMP_ID EMP_ADDRESS EMP_CONTACT

- This is the schema of the employee table. Schema defines the attributes of tables in the database. Schema is of three types: Physical schema, logical schema and view schema.




**In the following diagram, we have a schema that shows the relationship between three tables: Course, Student and Section. The diagram only shows the design of the database, it doesn't show the data present in those tables. Schema is only a structural view(design) of a database as shown in the diagram below.**

# Database Schema/Overall Structure of DBMS

- Physical Database Schema − This schema pertains to the actual storage of data and its form of storage like files, indices, etc. It defines how the data will be stored in a secondary storage.

- Logical Database Schema − This schema defines all the logical constraints that need to be applied on the data stored. It defines tables, views, and integrity constraints.

- External level : At this level, the user will get to see only the data stored in the database. Either they will see whole data values or any specific records. They will not have any information about how they are stored in the database.

# Database Schema/Overall Structure of DBMS

- Database schema is the skeleton of database. It is designed when the database doesn't exist at all.

- Once the database is operational, it is very difficult to make any changes to it. A database schema does not contain any data or information.

- Database instances tend to change with time.

- A DBMS ensures that its every instance (state) is in a valid state, by diligently following all the validations, constraints, and conditions that the database designers have imposed.

# DBMS Instance

**Definition of instance**: The data stored in database at a particular moment of time is called instance of database. Database schema defines the attributes in tables that belong to a particular database. The value of these attributes at a moment of time is called the instance of that database.

For example, we have seen the schema of table "employee" above. Let's see the table with the data now. At this moment the table contains two rows (records). This is the the current instance of the table "employee" because this is the data that is stored in this table at this particular moment of time.

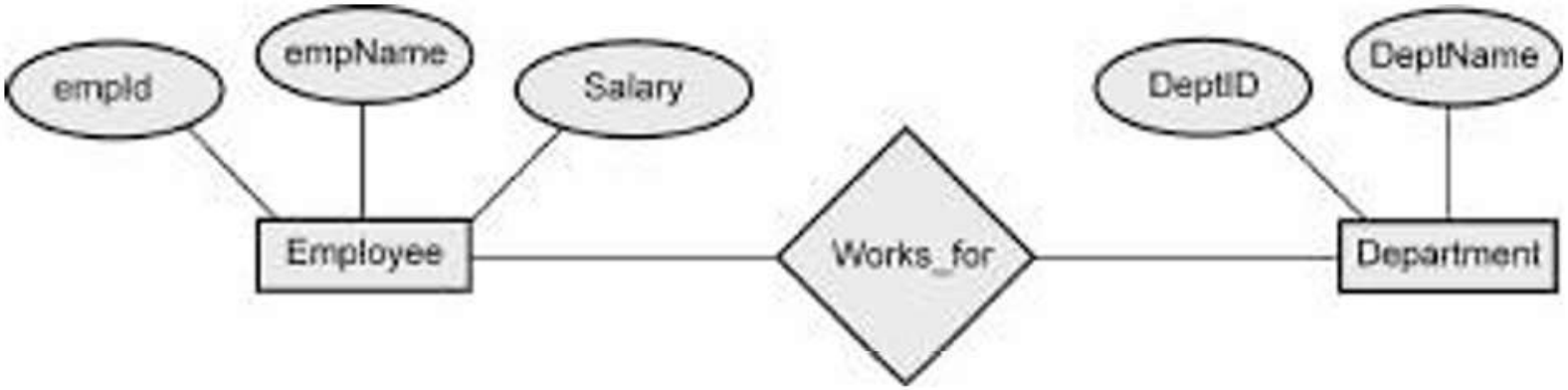
| EMP_NAME | EMP_ID | EMP_ADDRESS | EMP_CONTACT |
|---|---|---|---|
| Chaitanya | 101 | Noida | 95******** |
| Ajeet | 102 | Delhi | 99******** |

**Let's take another example:** Let's say we have a single table student in the database, today the table has 100 records, so today the instance of the database has 100 records. We are going to add another 100 records in this table by tomorrow so the instance of database tomorrow will have 200 records in table. In short, at a particular moment the data stored in database is called the instance, this changes over time as and when we add, delete or update data in the database.

# DBMS - Data Models

- Data models define how the logical structure of a database is modeled. Data Models are fundamental entities to introduce abstraction in a DBMS.

- Data models define how data is connected to each other and how they are processed and stored inside the system.




**For example - Following is a representation of Entity Relationship model in which the relationship works_for is between entities Employee and Department.**

# Entity-Relationship Model

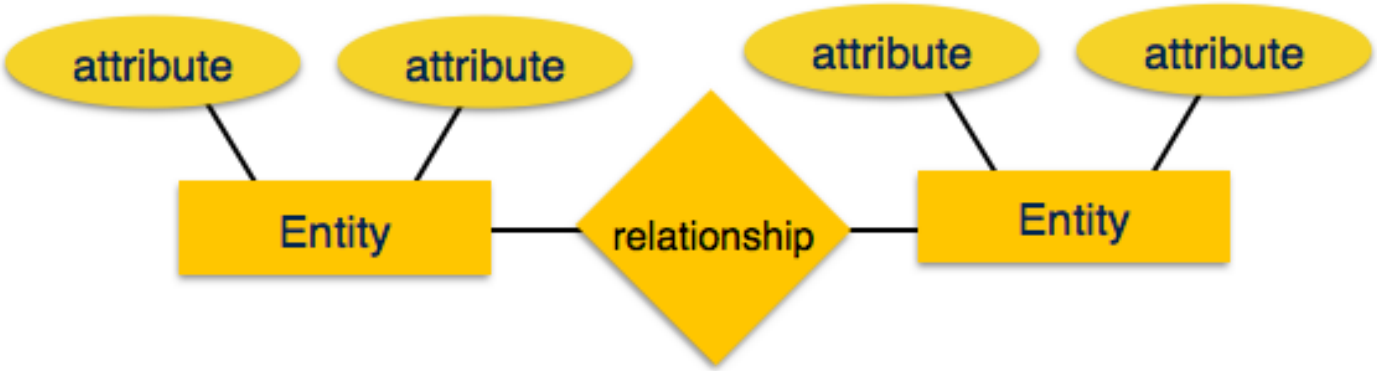
Entity-Relationship (ER) Model is based on the notion of real-world entities and relationships among them. While formulating real-world scenario into the database model, the ER Model creates entity set, relationship set, general attributes and constraints.

ER Model is best used for the conceptual design of a database.

ER Model is based on –

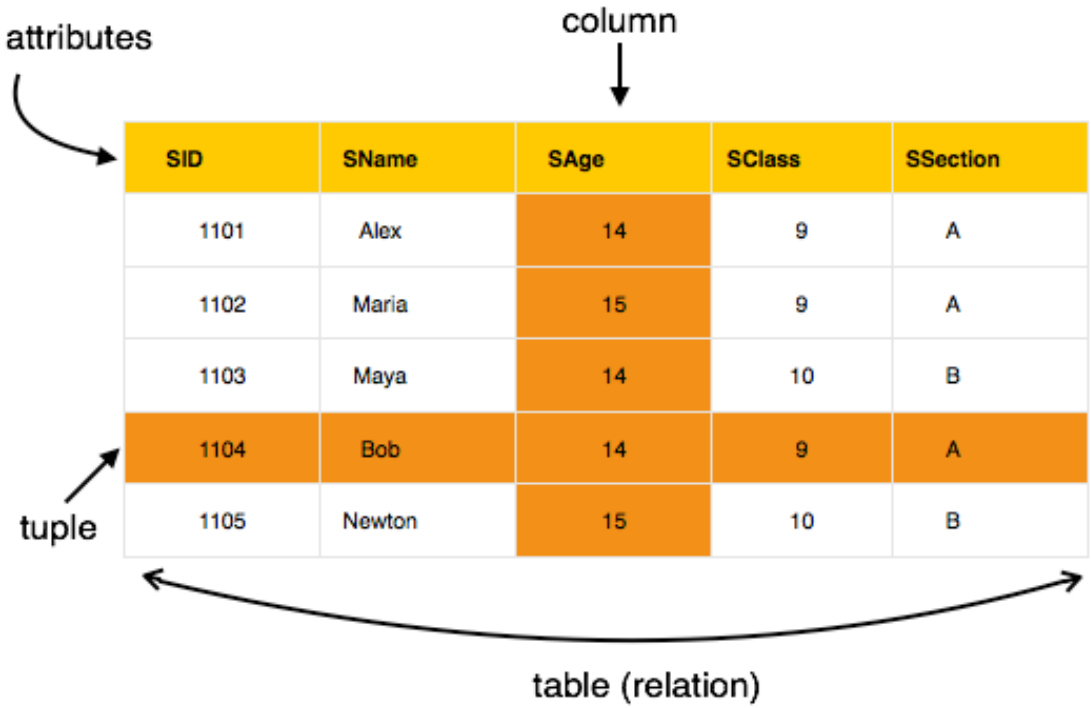
- **Entities** and their attributes.
- **Relationships** among entities.

These concepts are explained below.



# Relational Model

The most popular data model in DBMS is the Relational Model. It is more scientific a model than others. This model is based on first-order predicate logic and defines a table as an **n-ary relation**.




| SID | SName | SAge | SClass | SSection |
|---|---|---|---|---|
| 1101 | Alex | 14 | 9 | A |
| 1102 | Maria | 15 | 9 | A |
| 1103 | Maya | 14 | 10 | B |
| 1104 | Bob | 14 | 9 | A |
| 1105 | Newton | 15 | 10 | B |

**The main highlights of this model are –**

- **Data is stored in tables called relations.**
- **Relations can be normalized.**
- **In normalized relations, values saved are atomic values.**
- **Each row in a relation contains a unique value.**
- **Each column in a relation contains values from a same domain.**

# Data Abstraction/Data View in DBMS

- To ease the user interaction with database, the developers hide internal irrelevant details from users. This process of hiding irrelevant details from user is called data abstraction.

- For example: When you are booking a train ticket, you are not concerned how data is processing at the back end when you click “book ticket”, what processes are happening when you are doing online payments. You are just concerned about the message that pops up when your ticket is successfully booked. This doesn’t mean that the process happening at the back end is not relevant, it just means that you as a user are not concerned what is happening in the database.

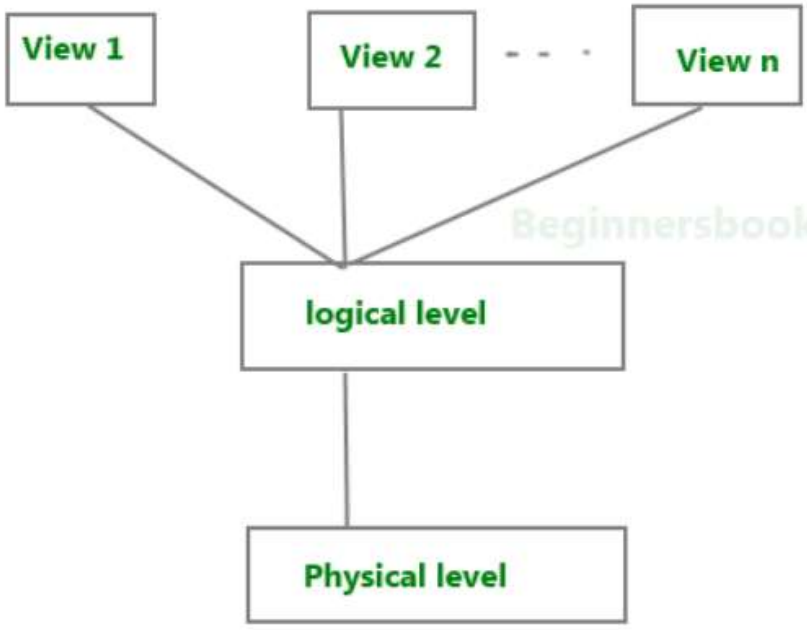
# Three levels of Data Abstraction

- **Physical**: This is the lowest level of data abstraction. It tells us how the data is actually stored in memory.
- Suppose we need to store the details of an employee. Blocks of storage and the amount of memory used for these purposes is kept hidden from the user.

- **Logical**: This level comprises of the information that is actually stored in the database in the form of tables. It also stores the relationship among the data entities in relatively simple structures.
- It describes structure and constraints for the entire database.

# Three levels of Data Abstraction

• **View:** This is the highest level of abstraction. Only a part of the actual database is viewed by the users. This level exists to ease the accessibility of the database by an individual user. Users view data in the form of rows and columns.




Three Levels of data abstraction

# Example

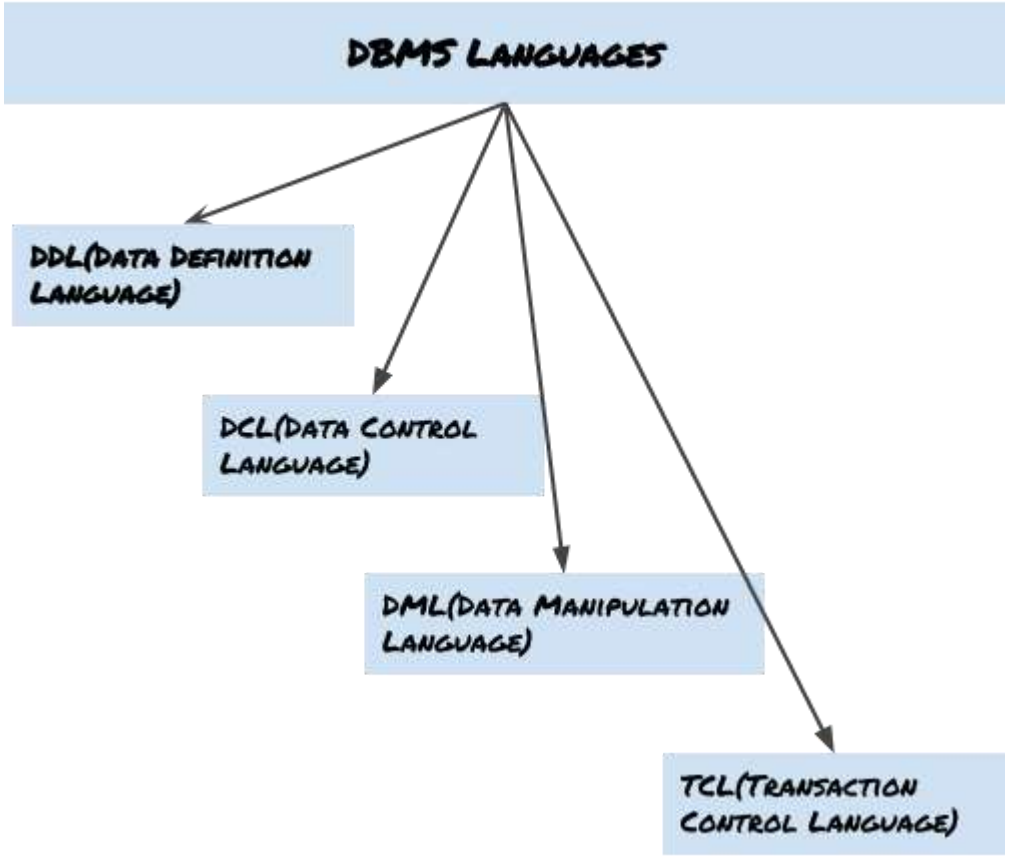
- Example: Let's say we are storing customer information in a customer table. At **physical** level these records can be described as blocks of storage (bytes, gigabytes, terabytes etc.) in memory. These details are often hidden from the programmers.

- At the **logical** level these records can be described as fields and attributes along with their data types, their relationship among each other can be logically implemented. The programmers generally work at this level because they are aware of such things about database systems.

- At **view** level, user just interact with system with the help of GUI and enter the details at the screen, they are not aware of how the data is stored and what data is stored; such details are hidden from them.

# DBMS languages

- Database languages are used to read, update and store data in a database. There are several such languages that can be used for this purpose; one of them is SQL (Structured Query Language).



## Data Definition Language (DDL)

DDL is used for specifying the database schema. It is used for creating tables, schema, indexes, constraints etc. in database. Lets see the operations that we can perform on database using DDL:

- To create the database instance – **CREATE**
- To alter the structure of database – **ALTER**
- To drop database instances – **DROP**
- To delete tables in a database instance – **TRUNCATE**
- To rename database instances – **RENAME**
- To drop objects from database such as tables – **DROP**
- To Comment – **Comment**

All of these commands either defines or update the database schema that's why they come under Data Definition language.

## Data Manipulation Language (DML)

DML is used for accessing and manipulating data in a database. The following operations on database comes under DML:

- To read records from table(s) – **SELECT**
- To insert record(s) into the table(s) – **INSERT**
- Update the data in table(s) – **UPDATE**
- Delete all the records from the table – **DELETE**

## Data Control language (DCL)

DCL is used for granting and revoking user access on a database –

- To grant access to user – GRANT
- To revoke access from user – REVOKE

**In practical data definition language, data manipulation language and data control languages are not separate language, rather they are the parts of a single database language such as SQL.**

## Transaction Control Language(TCL)

The changes in the database that we made using DML commands are either performed or rollbacked using TCL.

- To persist the changes made by DML commands in database – COMMIT
- To rollback the changes made to the database – ROLLBACK

# Data Independence

- A database system normally contains a lot of data in addition to users' data. For example, it stores data about data, known as metadata, to locate and retrieve data easily.

- It is rather difficult to update a set of metadata once it is stored in the database.

- But as a DBMS expands, it needs to change over time to satisfy the requirements of the users.

- If the entire data is dependent, it would become a tedious and highly complex job.

# Logical Data Independence

- Logical data is data about database, that is, it stores information about how data is managed inside.

- For example, a table (relation) stored in the database and all its constraints, applied on that relation.

- If we do some changes on table format, it should not change the data residing on the disk.

## Examples of changes under Logical Data Independence

- Due to Logical independence, any of the below change will not affect the external layer.

- Add/Modify/Delete a new attribute, entity or relationship is possible without a rewrite of existing application programs
- Merging two records into one
- Breaking an existing record into two or more records

# Physical Data Independence

- All the schemas are logical, and the actual data is stored in bit format on the disk.

- For example, in case we want to change or upgrade the storage system itself − suppose we want to replace hard-disks with SSD − it should not have any impact on the logical data or schemas.

- Compared to Logical Independence, it is easy to achieve physical data independence.

## Examples of changes under Physical Data Independence

- Due to Physical independence, any of the below change will not affect the conceptual layer.

➢ Using a new storage device like Hard Drive or Magnetic Tapes

➢ Modifying the file organization technique in the Database

➢ Switching to different data structures.

➢ Changing the access method.

➢ Modifying indexes.

➢ Changes to compression techniques or hashing algorithms.

➢ Change of Location of Database from say C drive to D Drive

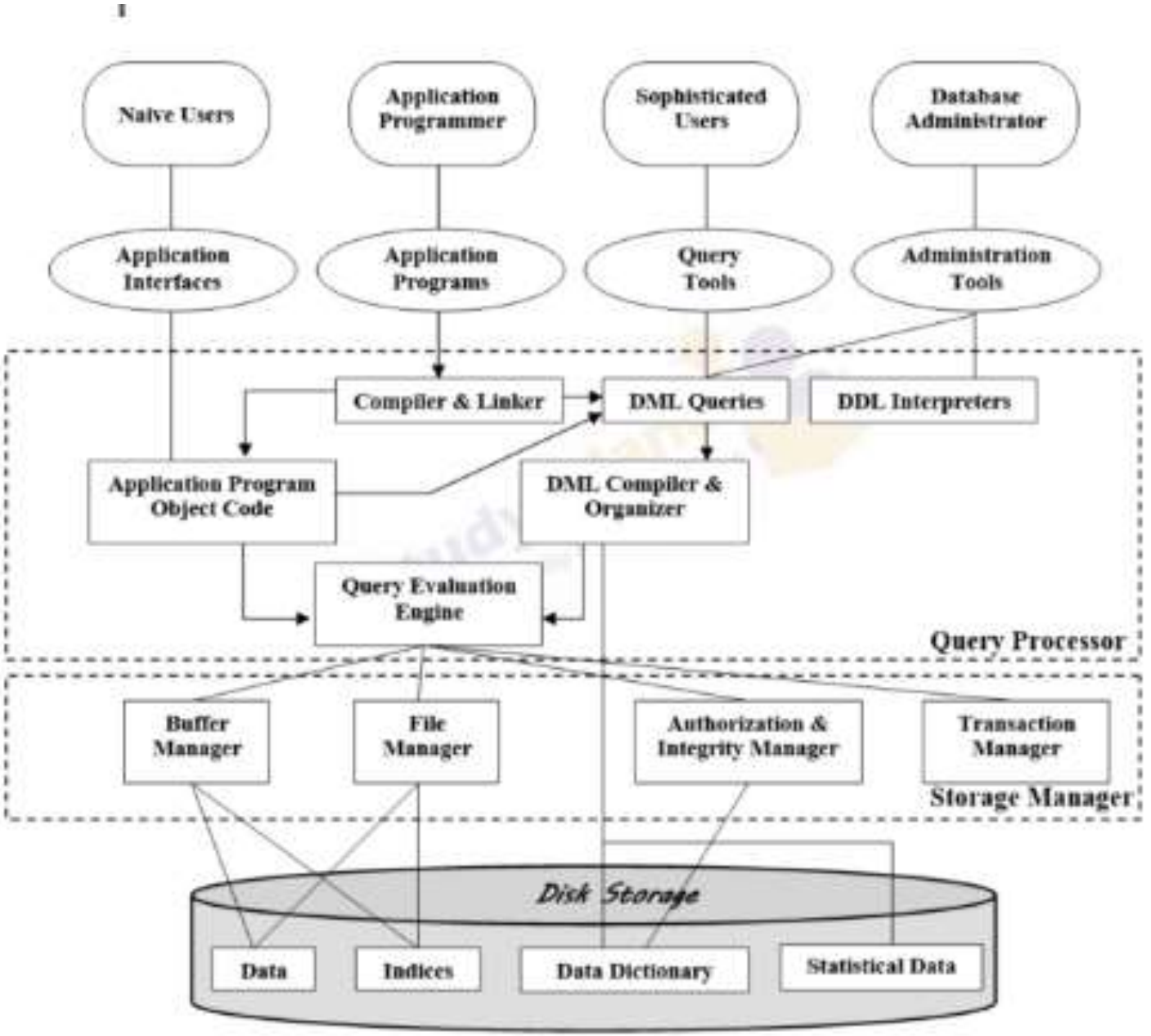
# Importance of Data Independence

- Helps you to improve the quality of the data.
- Database system maintenance becomes affordable.
- Enforcement of standards and improvement in database security
- Permit developers to focus on the general structure of the Database rather than worrying about the internal implementation
- Database incongruity is vastly reduced.
- Easily make modifications in the physical level is needed to improve the performance of the system.

# Difference between Physical and Logical Data Independence

| Logica Data Independence | Physical Data Independence |
| --- | --- |
| Logical Data Independence is mainly concerned with the structure or changing the data definition. | Mainly concerned with the storage of the data. |
| It is difficult as the retrieving of data is mainly dependent on the logical structure of data. | It is easy to retrieve. |
| Compared to Logic Physical independence it is difficult to achieve logical data independence. | Compared to Logical Independence it is easy to achieve physical data independence. |
| You need to make changes in the Application program if new fields are added or deleted from the database. | A change in the physical level usually does not need change at the Application program level. |
| Modification at the logical levels is significant whenever the logical structures of the database are changed. | Modifications made at the internal levels may or may not be needed to improve the performance of the structure. |
| Concerned with conceptual schema | Concerned with internal schema |
| Example: Add/Modify/Delete a new attribute | Example: change in compression techniques, hashing algorithms, storage devices, etc |

# DBMS ARCHITECTURE



# Types of Database Users

- **Naive users** are the unsophisticated who lacks DBMS knowledge but they frequently use the database applications in their daily life to get the desired results.
- **For example, Railway's ticket booking users, Clerks in any bank are naive user because they lacks DBMS knowledge but they still use the database.**

- **Application Programmers,** also known as Software Engineers, write backend code for application programs. They use languages like C, C++, JAVA, Python etc., to design, test, and maintain programs.

# Types of Database Users

- **Sophisticated Users** are those who have a deeper understanding of the DBMS. They can directly interact with the database using complex queries and tools. These database users in DBMS often include data analysts, scientists, and engineers who need detailed and flexible access to the data.

- Example: Accountants could employ SQL to sort through the numerous records of a particular company's sales database and produce highly reliable and detailed financial statements and projections. It enables them to pull out certain data and calculate and analyse trends all from a database.

# Database Administrators

- A database of any kind needs to be designed perfectly so that it should work without any issues. Once all the design is complete, it needs to be installed. Once this step is complete, users start using the database. The database grows as the data grows in the database. When the database becomes huge, its performance comes down. Also accessing the data from the database becomes challenge.

- A Database Administrator (DBA) is individual or person responsible for controlling, maintenance, coordinating, and operation of database management system.

# Responsibilities of DBA

- Installation, configuration and upgradation of databases like Microsoft SQL/ MySQL/ Oracle Server Software.

- Taking care of database design and implementation.

- Performance tuning: Since database is huge and it will have lots of tables, data, constraints and indices, there will be variations in the performance from time to time. Also, as data grows, the database will not work as expected. It is responsibility of the DBA to tune the database performance. He is responsible to make sure all the queries and programs works in fraction of seconds.

# Responsibilities of DBA

- Migrate database servers: – Sometimes, users using oracle would like to shift to SQL server or MySQL. It is the responsibility of DBA to make sure that migration happens without any failure, and there is no data loss.

- Backup and Recovery: – Proper backup and recovery programs needs to be developed by DBA and has to be maintained him. This is one of the main responsibilities of DBA. Database should be backed up regularly so that if there is any crash, it should be recovered without much effort and data loss.

# Responsibilities of DBA

- Security: – DBA is responsible for creating various database users and roles, and giving them different levels of access rights.

- Documentation: – DBA should be properly documenting all his activities so that if he quits or any new DBA comes in, he should be able to understand the database without any effort. He should basically maintain all his installation, backup, recovery, security methods. He should keep various reports about database performance.

## Query Processor

**DML Compiler:** It processes the DML statements into low level instruction (machine language), so that they can be executed.

**DDL Interpreter:** It processes the DDL statements into a set of table containing meta data (data about data).

**Query Optimizer:** The Query Optimizer executes instructions generated by the DML Compiler and improves query execution efficiency by choosing the best query plan, considering factors such as indexing, join order, and available system resources. For instance, if a query involves joining two large tables, the optimizer will select the **best join order to minimize query execution time**.

## Storage Manager

Authorization Manager: It ensures role-based access control, i.e,. checks whether the particular person is privileged to perform the requested operation or not.

Integrity Manager: It checks the integrity constraints when the database is modified.

Transaction Manager: It controls concurrent access by performing the operations in a scheduled way that it receives the transaction. Thus, it ensures that the database remains in the consistent state before and after the execution of a transaction.

File Manager: It manages the file space and the data structure used to represent information in the database.

Buffer Manager: It is responsible for cache memory and the transfer of data between the secondary storage and main memory.

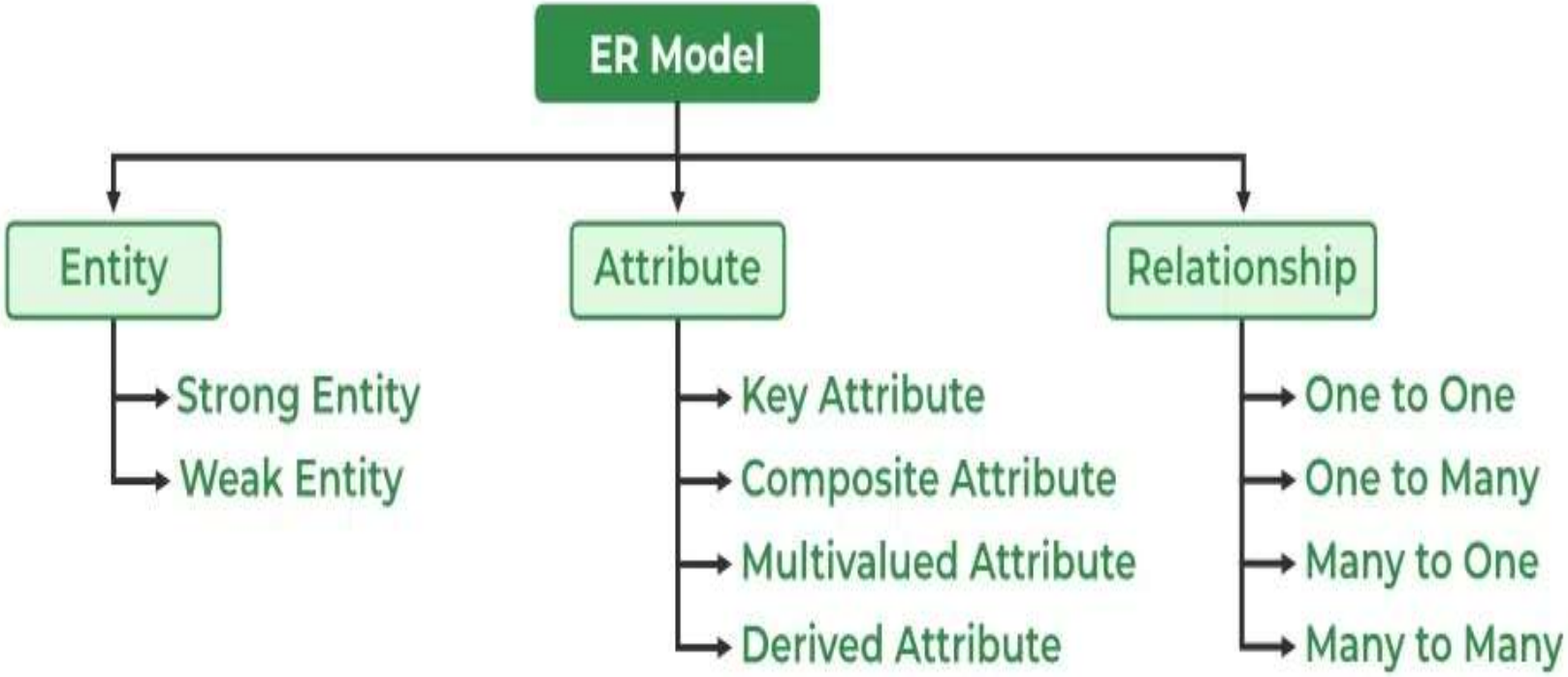
## Disk Storage

It contains the following essential components:

**Data Files:** It stores the actual data in the database.

**Data Dictionary:** It contains the information about the **structure of database objects** such as tables, constraints, and relationships.

**Indices:** Provides **faster data retrieval** by allowing the DBMS to find records quickly, improving query performance.


ER Model
Entity
Attribute
Relationship
Strong Entity
Weak Entity
Key Attribute
Composite Attribute
Multivalued Attribute
Derived Attribute
One to One
One to Many
Many to One
Many to Many

**What is an Entity?**

An Entity represents a real-world object, concept or thing about which data is stored in a database. It act as a building block of a database. Tables in relational database represent these entities.

Example of entities:

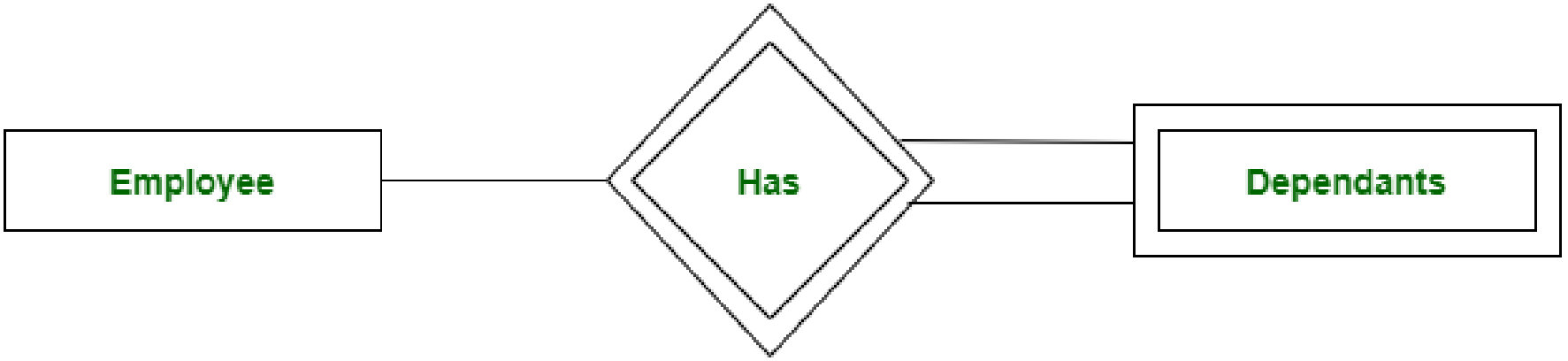
- **Real-World Objects:** Person, Car, Employee etc.
- **Concepts:** Course, Event, Reservation etc.
- **Things:** Product, Document, Device etc.

- For example, in a school database, students, teachers, classes, and courses offered can be considered as entities. All these entities have some attributes or properties that give them their identity.

# Weak Entity

**An entity that cannot be uniquely identified by its own attributes and relies on the relationship with other entity is called weak entity.** The weak entity is represented by a double rectangle. For example – a bank account cannot be uniquely identified without knowing the bank to which the account belongs, so bank account is a weak entity.

A company may store the information of dependents (Parents, Children, Spouse) of an Employee. But the dependents can't exist without the employee. So dependent will be a Weak Entity Type and Employee will be identifying entity type for dependent, which means it is Strong Entity Type.



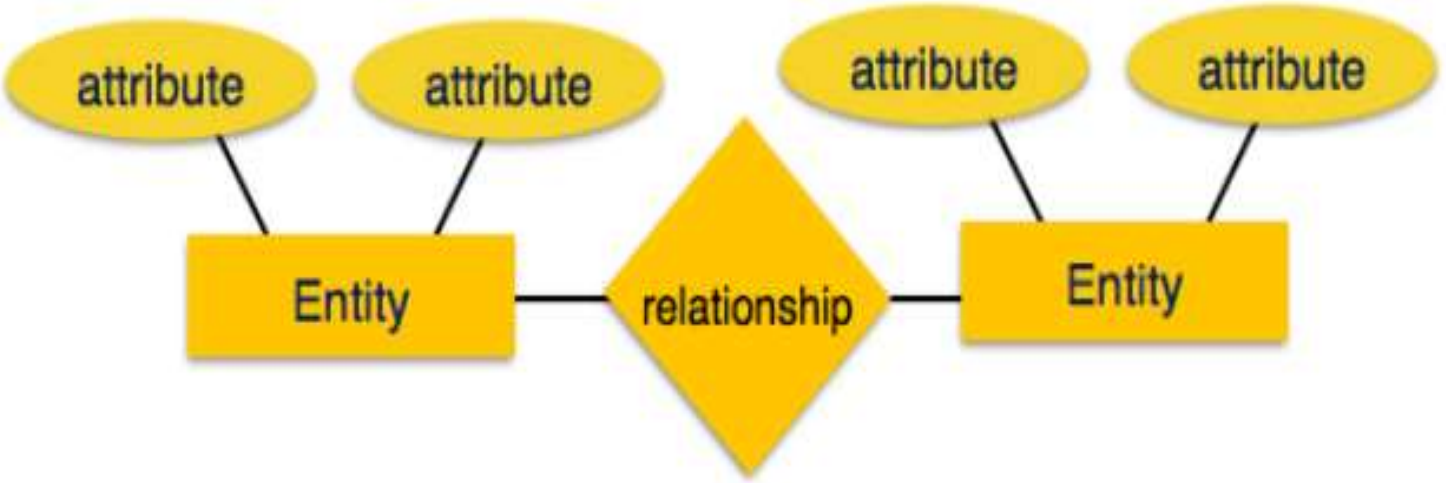
# Entity-Relationship (ER) Model

- An Entity–relationship model (ER model) describes the structure of a database with the help of a diagram, which is known as Entity Relationship Diagram (ER Diagram).

- An ER model is a design of a database that can later be implemented as a database.

- ER Model is based on:
  - Entities and their attributes.
  - Relationships among entities.

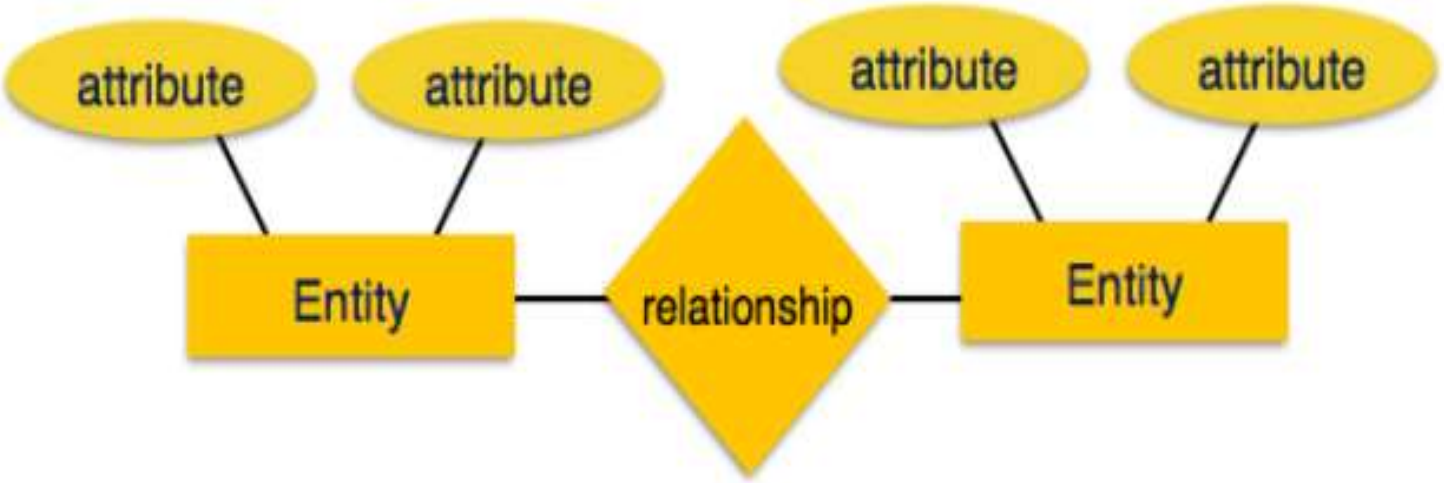
# Entity-Relationship (ER) Model




- **Entity** – An entity in an ER Model is a real-world object having properties called **attributes**. Every **attribute** is defined by its set of values called **domain**. For example, in a school database, a student is considered as an entity. Student has various attributes like name, age, class, etc.

- An entity may be any object, class, person or place. In the ER diagram, an entity can be represented as rectangles.

# Entity-Relationship (ER) Model




**Relationship** − A relationship is used to describe the relation between entities. Relationships are mapped with entities in various ways. Mapping cardinalities defines the relationship between two entities. **Diamond** is used to represent the relationship.

**Mapping cardinalities** −

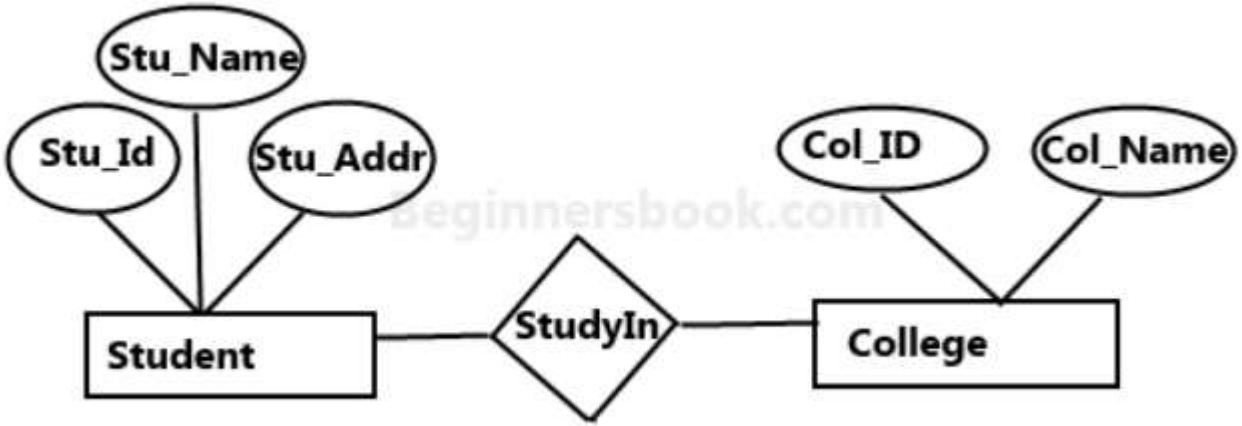
one to one

one to many

many to one

many to many

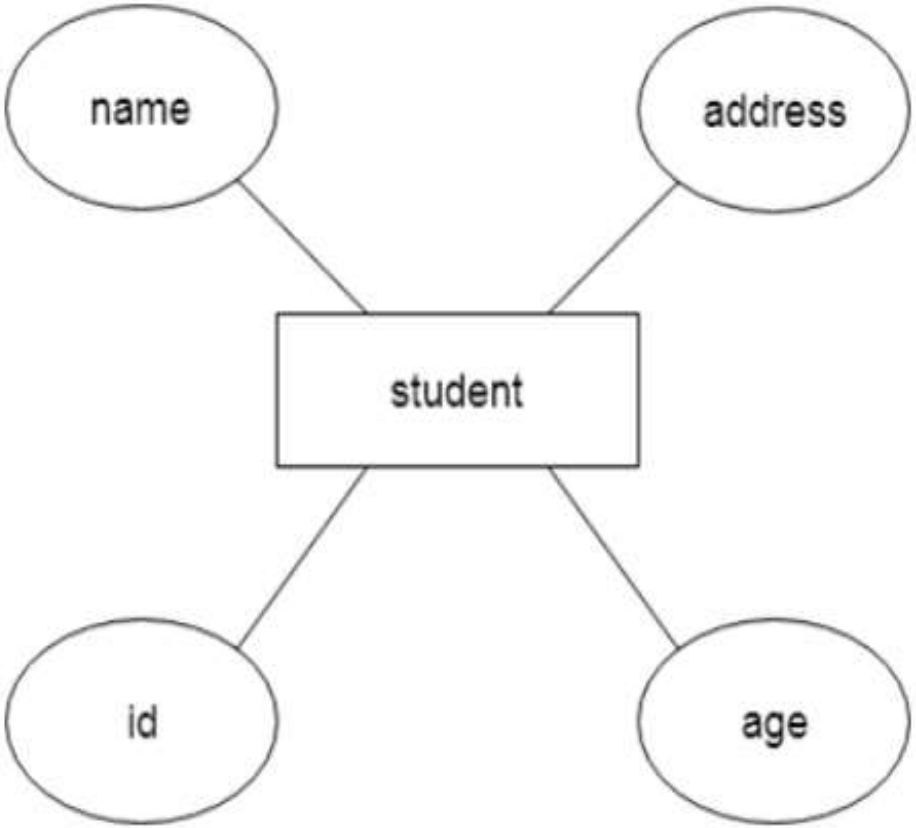
# Entity-Relationship (ER) Model Example



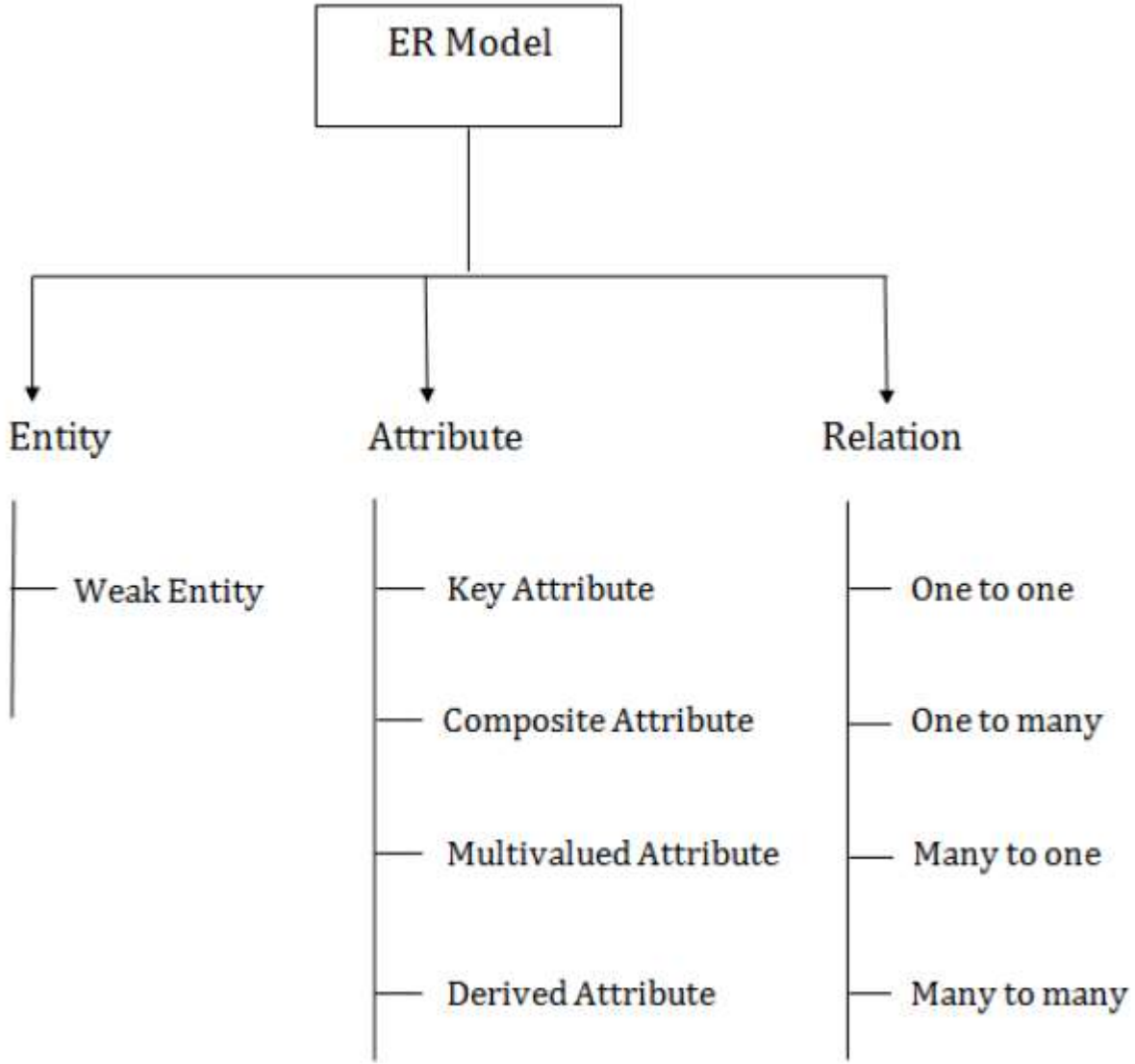

**Rectangle**: Represents Entity sets.
**Ellipses**: Attributes
**Diamonds**: Relationship Set
**Lines**: They link attributes to Entity Sets and Entity sets to Relationship Set

# Entity-Relationship (ER) Model Example

**For example,** Suppose we design a school database. In this database, the student will be an entity with attributes like address, name, id, age, etc. The address can be another entity with attributes like city, street name, pin code, etc and there will be a relationship between them.



# Components of ER Diagram



# Weak Entity

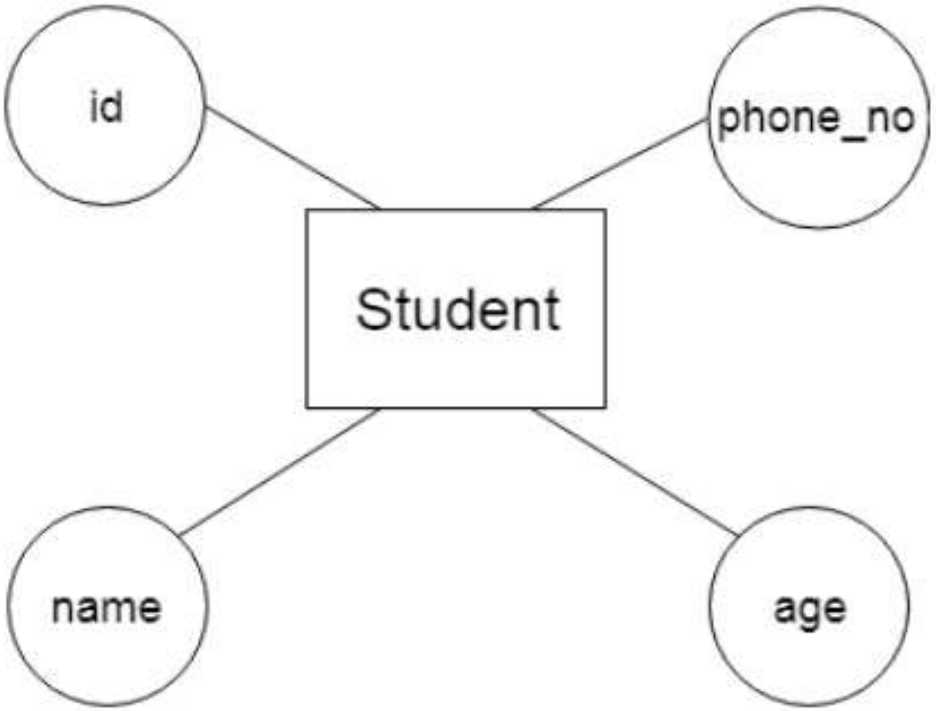
- An entity that cannot be uniquely identified by its own attributes and relies on the relationship with other entity is called weak entity. The weak entity is represented by a double rectangle. For example – a bank account cannot be uniquely identified without knowing the bank to which the account belongs, so bank account is a weak entity.



# Attribute

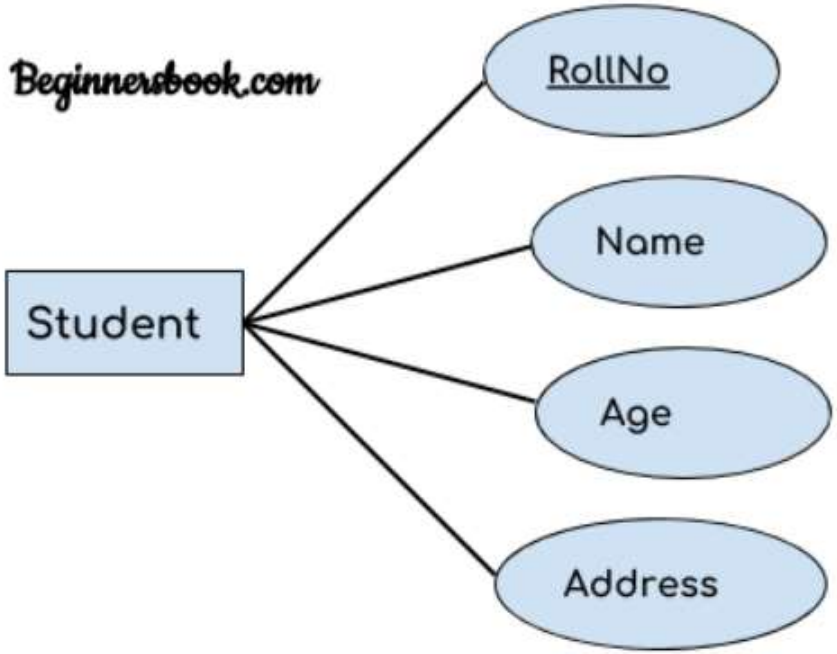
- The attribute is used to describe the property of an entity. Eclipse is used to represent an attribute.

**For example,** id, age, contact number, name, etc. can be attributes of a student.



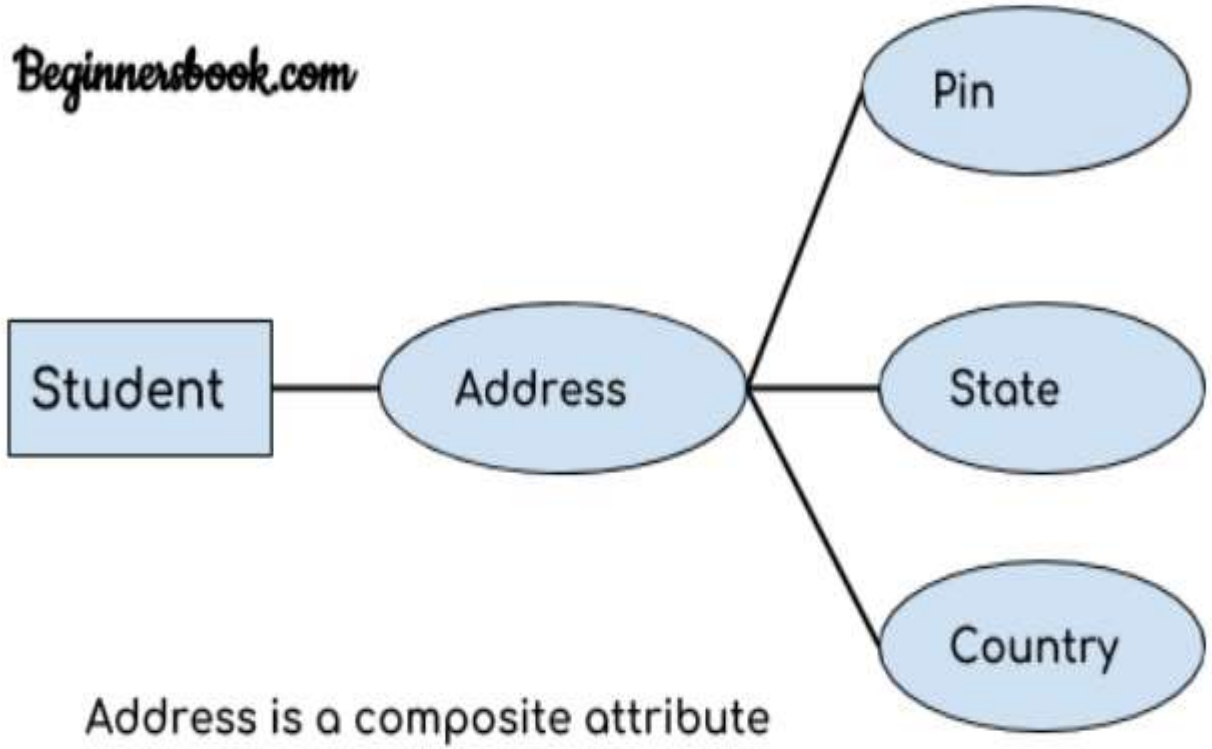
# Key Attribute

- A key attribute can uniquely identify an entity from an entity set. It represents a primary key. The key attribute is represented by an ellipse with the text underlined.
- For example, student roll number can uniquely identify a student from a set of students.



# Composite Attribute

An attribute that is a combination of other attributes is known as composite attribute. For example, In student entity, the student address is a composite attribute as an address is composed of other attributes such as pin code, state, country.

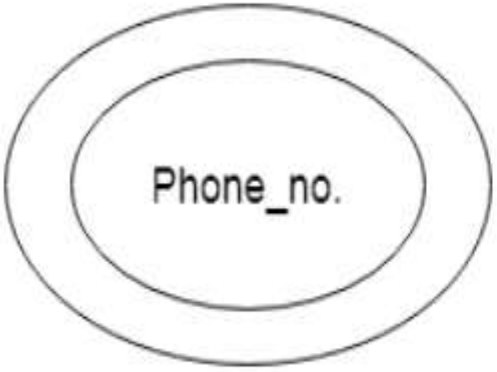



Address is a composite attribute

# Multivalued Attribute

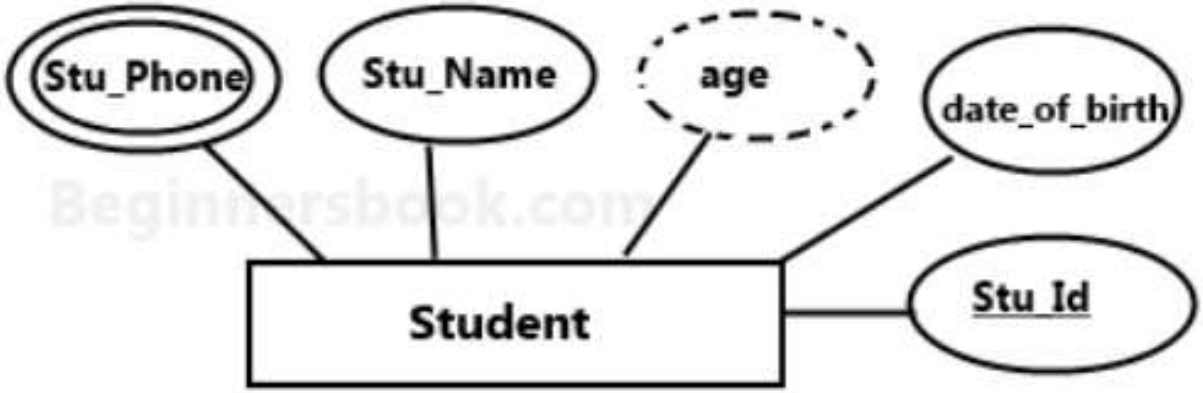
An attribute can have more than one value. These attributes are known as a multivalued attribute. The double oval is used to represent multivalued attribute.

**For example,** a student can have more than one phone number.



# Derived Attribute

- A derived attribute is one whose value is dynamic and derived from another attribute. It is represented by dashed oval in an ER Diagram. For example – Person age is a derived attribute as it changes over time and can be derived from another attribute (Date of birth).



# Relationship

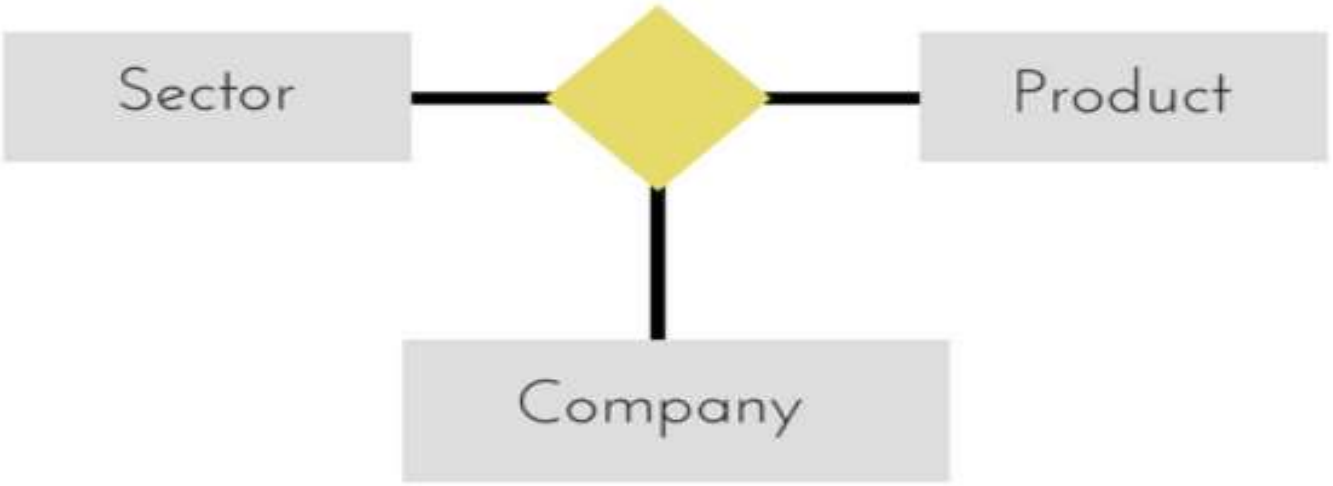
- A relationship is represented by diamond shape in ER diagram, it shows the relationship among entities.
- A relationship where two entities are participating is called a binary relationship. Cardinality is the number of instance of an entity from a relation that can be associated with the relation.



# Ternary Relationship

Relationship of degree three is called Ternary relationship.

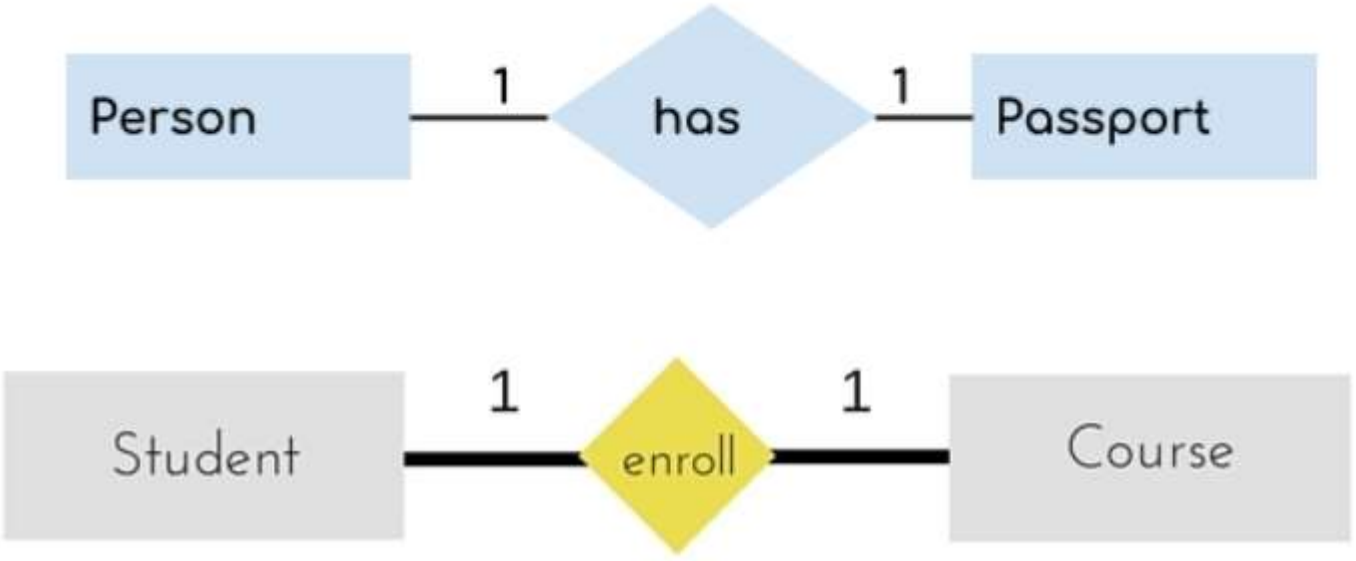
A Ternary relationship involves three entities. In such relationships we always consider two entites together and then look upon the third.




- The above relationship involves 3 entities.
- Company operates in Sector, producing some Products.

# One to One Relationship

- When a single instance of an entity is associated with a single instance of another entity then it is called one to one relationship. For example, a person has only one passport.



# Many-to-one relationship

When more than one instance of the entity on the left, and only one instance of an entity on the right associates with the relationship then it is known as a many-to-one relationship.

**For example,** Student enrolls for only one course, but a course can have many students.




For example – many students can study in a single college but a college can have many students at the same time.




Beginnersbook.com

# Many-to-many relationship

When more than one instance of the entity on the left, and more than one instance of an entity on the right associates with the relationship then it is known as a many-to-many relationship.

**For example,** Employee can assign by many projects and project can have many employees.




For example, Students as a group are associated with multiple faculty members, and faculty members can be associated with multiple students.



# SQL

# SQL Constraints

# SQL Constraints

- SQL constraints are used to specify rules for data in a table.

- Constraints are the rules that we can apply on the type of data in a table.

- This ensures the accuracy and reliability of the data in the database.

# SQL Constraints

- Constraints could be either on a column level or a table level.

- We can create constraints on single or multiple columns of any table. It maintain the data integrity of the table/database.

- For example postal code of India consists of size six digits. no below or no more then six digit should be allowed to enter.

# SQL Constraints

- Following are the most used constraints that can be applied to a table.

➢ NOT NULL

➢ UNIQUE

➢ PRIMARY KEY

➢ FOREIGN KEY

➢ CHECK

➢ DEFAULT

# NOT NULL

- NOT NULL constraint restricts a column from having a NULL value.

- Once NOT NULL constraint is applied to a column, you cannot pass a null value to that column.

- It enforces a column to contain a proper value.

- Example using NOT NULL constraint

➢ CREATE TABLE Student(Roll_no int NOT NULL, Name varchar(60), Age int);

# NOT NULL

- **NOT NULL Constraint on ALTER TABLE**
- Write a query to alter the table named Students, where a new column of DOB must be added, and it should not have any NULL values.

- ALTER TABLE Student ADD COLUMN DOB year NOT NULL;

# UNIQUE

- This constraint tells that all the values in the column must be unique. That is, the values in any row of a column must not be repeated.

- CREATE TABLE Student(Roll_no int NOT NULL UNIQUE, Name varchar(60), Age int);

# UNIQUE

- **Using UNIQUE constraint after Table is created (Column Level)**
- ALTER TABLE Student ADD UNIQUE(Roll_no);
- The above query specifies that Roll_no field of Student table will only have unique value.

# PRIMARY KEY

- A value in specified column must be unique for each row in a table and not a NULL. Primary key used to identify individual records.

- **Using PRIMARY KEY constraint at Table Level**

➢CREATE table Student (Roll_no int PRIMARY KEY, Name varchar(60) NOT NULL, Age int);

# PRIMARY KEY

- **Using PRIMARY KEY constraint at Column Level**

➢ ALTER table Student ADD PRIMARY KEY (Roll_no);

- The above command will creates a PRIMARY KEY on the Roll_no.

# Unique key v/s Primary key

| | |
|---|---|
| A table can have more than one unique key. | Only one primary key is allowed to use in a table. |
| Unique key constraints can accept **only one NULL value** for column. | The primary key does not accept NULL values. |

# FOREIGN KEY

- A FOREIGN KEY is a key used to link two tables together.

- A FOREIGN KEY is a field in one table that refers to the PRIMARY KEY in another table.

- The table containing the foreign key is called the child table, and the table containing the primary key is called the parent table.

Look at the following two tables:

"Persons" table:

| PersonID | LastName | FirstName | Age |
|---|---|---|---|
| 1 | Hansen | Ola | 30 |
| 2 | Svendson | Tove | 23 |
| 3 | Pettersen | Kari | 20 |

"Orders" table:

| OrderID | OrderNumber | PersonID |
|---|---|---|
| 1 | 77895 | 3 |
| 2 | 44678 | 3 |
| 3 | 22456 | 2 |
| 4 | 24562 | 1 |

# FOREIGN KEY

- The following SQL query creates a FOREIGN KEY on the "PersonID" column when the "Orders" table is created:

- CREATE TABLE Orders (
  OrderID int NOT NULL,
  OrderNumber int NOT NULL,
  PersonID int,
  PRIMARY KEY (OrderID),
  FOREIGN KEY (PersonID) REFERENCES Persons(Pers
onID));

# SQL CHECK Constraint

- The CHECK constraint makes sure that all values in a column satisfy a specific condition.

- If you define a CHECK constraint on a single column it allows only certain values for this column.

# SQL CHECK Constraint

- For example, the following query creates a table called CUSTOMERS and adds three columns. Here, we add a CHECK with AGE column, so that you cannot have any CUSTOMER who is below 18 years.

- CREATE TABLE CUSTOMERS(

ID INT NOT NULL,

NAME VARCHAR (20) NOT NULL,

AGE INT NOT NULL **CHECK (AGE >= 18)**,

PRIMARY KEY (ID));

# SQL CHECK Constraint

- If the CUSTOMERS table has already been created, then to add a CHECK constraint to AGE column, we need to write following query:

- ALTER TABLE CUSTOMERS

MODIFY AGE INT NOT NULL CHECK (AGE >= 18 );

# DEFAULT

- The DEFAULT constraint is used to provide a default value for a column.
- The default value will be added to all new records IF no other value is specified.
- The following SQL sets a DEFAULT value for the "City" column when the "Persons" table is created:
- CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Age int,
  City varchar(255) DEFAULT ’Mumbai’);